Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ»

КАФЕДРА «СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ»

Рубежный контроль №1

ДИСЦИПЛИНА: «Парадигмы и конструкции языков программирования»

Выполнил: студент гр. ИУ5-34Б

_____ (Лимаренко Д. А.)
(Подпись) (Ф.И. О.)

2026 г.

Задание:

1) Необходимо создать два класса данных в соответствии с Вашим вариантом предметной области, которые связаны отношениями один-ко-многим и многие-ко многим.

Пример классов данных для предметной области Сотрудник-Отдел:

1. Класс «Сотрудник», содержащий поля:

● ID записи о сотруднике;

● Фамилия сотрудника;

● Зарплата (количественный признак);

● ID записи об отделе. (для реализации связи один-ко-многим)

2. Класс «Отдел», содержащий поля:

● ID записи об отделе;

● Наименование отдела.

3. (Для реализации связи многие-ко-многим) Класс «Сотрудники отдела», содержащий поля:

● ID записи о сотруднике;

● ID записи об отделе.

2) Необходимо создать списки объектов классов, содержащих тестовые данные (3-5 записей), таким образом, чтобы первичные и вторичные ключи соответствующих записей были связаны по идентификаторам.

3) Необходимо разработать запросы в соответствии с Вашим вариантом. Запросы сформулированы в терминах классов «Сотрудник» и «Отдел», которые используются в примере. Вам нужно перенести эти требования в Ваш вариант предметной области. При разработке запросов необходимо по возможности использовать функциональные возможности языка Python (list/dict comprehensions, функции высших порядков).

Для реализации запроса №2 введите в класс, находящийся на стороне связи «много», произвольный количественный признак, например, «зарплата сотрудника».

Результатом рубежного контроля является документ в формате PDF, который содержит текст программы и результаты ее выполнения.

Вариант запросов: Г

Предметная область №1 «Студент – Группа»

Выполнение работы

1. Описание классов и связей

Класс Student хранит идентификатор, фамилию, стипендию и идентификатор основной группы. Класс Group хранит идентификатор и название группы. Класс StudentGroup связывает идентификаторы студентов и групп для отношения многие-ко-многим.

Связь один-ко-многим реализована полем Student.group_id. Связь многие-ко-многим реализована списком объектов StudentGroup. Для проверки подготовлено 5 студентов и 4 группы.

2. Исходный код программы

```
from operator import itemgetter
class Student:
    """Студент учебной группы."""
    def __init__(self, id, surname, scholarship, group_id):
        self.id = id
        self.surname = surname
        self.scholarship = scholarship
        self.group_id = group_id
class Group:
    """Учебная группа."""
    def __init__(self, id, name):
        self.id = id
        self.name = name
class StudentGroup:
    """Связь студентов и групп для отношения многие-ко-многим."""
    def __init__(self, student_id, group_id):
        self.student_id = student_id
        self.group_id = group_id
groups = [
    Group(1, "Аналитика-01"),
    Group(2, "Архитектура-02"),
    Group(3, "ИУ5-34"),
    Group(4, "Программирование-04"),
]
students = [
    Student(1, "Иванов", 2500, 1),
    Student(2, "Петров", 3200, 1),
    Student(3, "Сидорова", 1800, 2),
    Student(4, "Кузнецов", 3200, 3),
    Student(5, "Смирнова", 2200, 3),
]
student_groups = [
    StudentGroup(1, 1),
    StudentGroup(1, 3),
    StudentGroup(2, 1),
    StudentGroup(2, 2),
    StudentGroup(3, 2),
    StudentGroup(3, 4),
    StudentGroup(4, 3),
    StudentGroup(5, 3),
    StudentGroup(5, 4),
]
def get_query_results():
```

```
    """Выполнение запросов варианта Г."""
    # Соединение данных один-ко-многим по полю group_id.
    one_to_many = [
        (group.name, student.surname, student.scholarship)
        for group in groups
        for student in students
        if student.group_id == group.id
    ]
    # Соединение данных многие-ко-многим через класс StudentGroup.
    many_to_many_temp = [
        (group.name, link.student_id)
        for group in groups
        for link in student_groups
        if group.id == link.group_id
    ]
    many_to_many = [
        (student.surname, group_name)
        for group_name, student_id in many_to_many_temp
        for student in students
        if student.id == student_id
    ]
    groups_starting_with_a = list(filter(lambda group: group.name.startswith("A"), groups))
    res_1 = {
        group.name: [student.surname for student in students if student.group_id == group.id]
        for group in groups_starting_with_a
    }
    max_scholarship_by_group = []
    for group in groups:
        group_students = list(filter(lambda row: row[0] == group.name, one_to_many))
        if len(group_students) > 0:
            scholarships = [scholarship for _, _, scholarship in group_students]
            max_scholarship = max(scholarships)
            max_scholarship_by_group.append((group.name, max_scholarship))
    res_2 = sorted(max_scholarship_by_group, key=itemgetter(1), reverse=True)
    res_3 = sorted(many_to_many, key=itemgetter(1))
    return res_1, res_2, res_3
def main():
    """Вывод результатов запросов варианта Г."""
    res_1, res_2, res_3 = get_query_results()
    print("Задание Г1")
    print(res_1)
    print("\nЗадание Г2")
    print(res_2)
    print("\nЗадание Г3")
    print(res_3)
if __name__ == "__main__":
    main()
```

3. Результаты выполнения программы

Команда запуска: python3 main.py

Задание Г1

{'Аналитика-01': ['Иванов', 'Петров'], 'Архитектура-02': ['Сидорова']}

Задание Г2

[('Аналитика-01', 3200), ('ИУ5-34', 3200), ('Архитектура-02', 1800)]

Задание Г3

[('Иванов', 'Аналитика-01'), ('Петров', 'Аналитика-01'), ('Петров', 'Архитектура-02'), ('Сидорова', 'Архитектура-02'), ('Иванов', 'ИУ5-34'), ('Кузнецов', 'ИУ5-34'), ('Смирнова', 'ИУ5-34'), ('Сидорова', 'Программирование-04'), ('Смирнова', 'Программирование-04')]

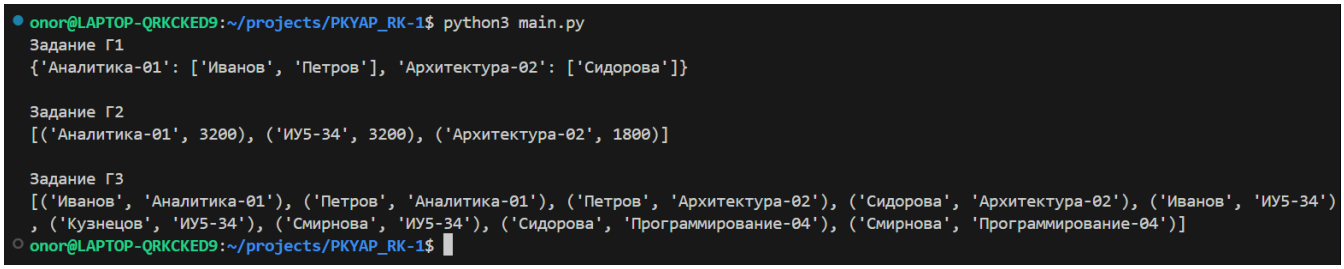


Рисунок 1 – Результат выполнения программы

4. Модульное тестирование

Созданы пять модульных тестов, проверяющих идентификаторы объектов, выбор групп на букву «А», обработку пустой группы, максимальную стипендию и корректность сортировки связей многие-ко-многим.

Команда запуска: python3 -m unittest -v

test_data_contains_five_students_four_groups_and_valid_ids ... ok

test_one_to_many_query_selects_a_groups_and_students ... ok

test_one_to_many_query_keeps_empty_a_group ... ok

test_max_scholarship_skips_empty_groups_and_sorts_descending ... ok

test_many_to_many_query_contains_all_links_sorted_by_group_name ... ok

Ran 5 tests

OK

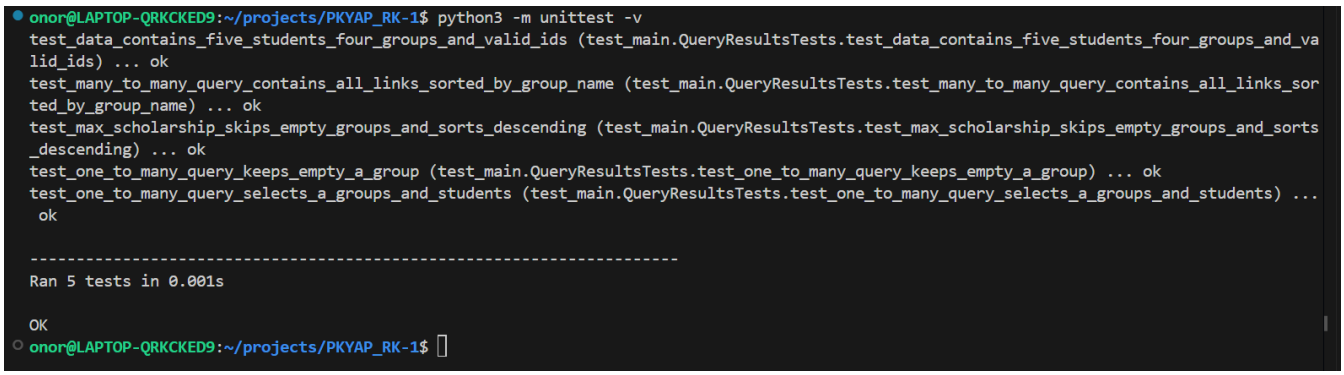


Рисунок 2 – Результаты модульного тестирования

Вывод

В ходе рубежного контроля разработана программа на языке Python для предметной области «Студент — Группа». Реализованы связи один-ко-многим и многие-ко-многим, выполнены запросы варианта Г с использованием генераторов списков, функций filter, max и sorted. Проверки подтверждают корректность результатов на подготовленных тестовых данных.